

Phần 5: Query Methods và JPQL

5.1 Query Methods (Derived Queries)

Spring Data JPA có khả năng mạnh mẽ trong việc tự động tạo ra các triển khai (implementation) cho các phương thức repository chỉ dựa trên tên của phương thức. Đây được gọi là "Query Methods" hoặc "Derived Queries".

5.1.1 Cú pháp cơ bản

Spring Data JPA phân tích tên phương thức và chuyển đổi nó thành một truy vấn JPQL (Java Persistence Query Language) hoặc SQL tương ứng. Cú pháp tuân theo một quy ước đặt tên nhất định.

Java

```
public interface UserRepository extends JpaRepository<User, Long> {

    // Find by single property: Tìm User theo một thuộc tính duy nhất.
    List<User> findByName(String name); // Tìm tất cả User có tên khớp với 'name'.
    User findByEmail(String email); // Tìm một User duy nhất có email khớp với 'email'.

    // Find by multiple properties: Tìm User theo nhiều thuộc tính kết hợp.
    List<User> findByNameAndEmail(String name, String email); // Tìm User có tên là 'name' VÀ email là 'email'.
    List<User> findByNameOrEmail(String name, String email); // Tìm User có tên là 'name' HOẶC email là 'email'.

    // Comparison operators: Các toán tử so sánh.
    List<User> findByAgeGreaterThan(Integer age); // Tìm User có tuổi lớn hơn 'age'.
    List<User> findByAgeGreaterThanEqual(Integer age); // Tìm User có tuổi lớn hơn hoặc bằng 'age'.
    List<User> findByAgeLessThan(Integer age); // Tìm User có tuổi nhỏ hơn 'age'.
    List<User> findByAgeLessThanEqual(Integer age); // Tìm User có tuổi nhỏ hơn hoặc bằng 'age'.
    List<User> findByAgeBetween(Integer startAge, Integer endAge); // Tìm User có tuổi nằm trong khoảng từ 'startAge' đến 'endAge' (bao gồm cả hai).

    // String operations: Các thao tác với chuỗi.
    List<User> findByNameContaining(String name); // Tìm User có tên chứa
```

chuỗi 'name' (tương đương SQL LIKE %name%).

`List<User> findByNameStartingWith(String prefix);` // Tìm User có tên bắt đầu bằng 'prefix' (tương đương SQL LIKE prefix%).

`List<User> findByNameEndingWith(String suffix);` // Tìm User có tên kết thúc bằng 'suffix' (tương đương SQL LIKE %suffix%).

`List<User> findByNameIgnoreCase(String name);` // Tìm User có tên khớp với 'name', không phân biệt chữ hoa chữ thường.

`List<User> findByNameContainingIgnoreCase(String name);` // Tìm User có tên chứa 'name', không phân biệt chữ hoa chữ thường.

// Null checks: Kiểm tra giá trị null.

`List<User> findByEmailIsNull();` // Tìm User có email là NULL.

`List<User> findByEmailIsNotNull();` // Tìm User có email KHÔNG phải là NULL.

// Collection operations: Các thao tác với tập hợp.

`List<User> findByRolesIn(Collection<Role> roles);` // Tìm User có ít nhất một Role nằm trong tập hợp 'roles' đã cho.

`List<User> findByRolesNotIn(Collection<Role> roles);` // Tìm User không có Role nào nằm trong tập hợp 'roles' đã cho.

// Boolean operations: Các thao tác với giá trị boolean.

`List<User> findByActiveTrue();` // Tìm User có thuộc tính 'active' là TRUE.

`List<User> findByActiveFalse();` // Tìm User có thuộc tính 'active' là FALSE.

// Date operations: Các thao tác với ngày tháng.

`List<User> findByCreatedDateAfter(Date date);` // Tìm User được tạo sau ngày 'date'.

`List<User> findByCreatedDateBefore(Date date);` // Tìm User được tạo trước ngày 'date'.

}

Giải thích:

- **Quy ước đặt tên:** Spring Data JPA tuân theo một quy ước đặt tên chặt chẽ. Tên phương thức bắt đầu bằng `find`, `read`, `get`, `query`, `stream`, `count`, `exists`, `delete`, `remove` theo sau là `By` và tên thuộc tính của Entity.
- **Kết hợp điều kiện:** Bạn có thể kết hợp nhiều điều kiện bằng cách sử dụng `And` và `Or` trong tên phương thức.
- **Toán tử:** Spring Data JPA hỗ trợ nhiều toán tử so sánh như `GreaterThan`, `LessThan`, `Between`, `Like`, `Containing`, `StartingWith`, `EndingWith`, `IsNull`, `IsNotNull`, `In`,

NotIn , True , False , After , Before .

- **Không phân biệt chữ hoa/thường:** Sử dụng `IgnoreCase` để thực hiện tìm kiếm không phân biệt chữ hoa/thường cho các thuộc tính kiểu chuỗi.

5.1.2 Limiting Results

Bạn có thể giới hạn số lượng kết quả trả về từ một truy vấn bằng cách sử dụng các từ khóa như `First` hoặc `Top` trong tên phương thức. Bạn cũng có thể chỉ định số lượng kết quả cụ thể.

Java

```
public interface UserRepository extends JpaRepository<User, Long> {

    // Limit results: Giới hạn số lượng kết quả trả về.
    User findFirstByNameAsc(); // Tìm User đầu tiên khi sắp xếp theo
    // tên tăng dần.
    User findTopByOrderByAgeDesc(); // Tìm User có tuổi cao nhất.
    List<User> findFirst10ByNameAsc(); // Tìm 10 User đầu tiên khi sắp
    // xếp theo tên tăng dần.
    List<User> findTop5ByAgeGreaterThan(Integer age); // Tìm 5 User có tuổi
    // lớn hơn 'age' đầu tiên.

    // Distinct results: Trả về các kết quả duy nhất.
    List<User> findDistinctByName(String name); // Tìm các User có tên duy
    // nhất khớp với 'name'.

    // Exists queries: Kiểm tra sự tồn tại của một Entity.
    boolean existsByEmail(String email); // Kiểm tra xem có User nào có email
    // này không.

    // Count queries: Đếm số lượng Entity thỏa mãn điều kiện.
    long countByStatus(String status); // Đếm số lượng User có trạng thái
    // 'status'.

    // Delete queries: Xóa các Entity thỏa mãn điều kiện.
    void deleteByStatus(String status); // Xóa tất cả User có trạng thái
    // 'status'.
    List<User> removeByStatus(String status); // Xóa tất cả User có trạng
    // thái 'status' và trả về danh sách các User đã xóa.
}
```

Giải thích:

- `First` và `Top` : Các từ khóa này được sử dụng để giới hạn số lượng kết quả trả về. `First` và `Top` có thể được sử dụng thay thế cho nhau.
- `Distinct` : Đảm bảo rằng các kết quả trả về là duy nhất.
- `existsBy...` : Trả về `true` nếu có ít nhất một Entity thỏa mãn điều kiện, `false` nếu không có. Rất hiệu quả để kiểm tra sự tồn tại mà không cần tải toàn bộ dữ liệu.
- `countBy...` : Trả về số lượng Entity thỏa mãn điều kiện.
- `deleteBy...` và `removeBy...` : Thực hiện thao tác xóa. `removeBy...` sẽ trả về danh sách các Entity đã bị xóa, trong khi `deleteBy...` thì không.

5.1.3 Sorting và Pagination

Spring Data JPA cung cấp các cách dễ dàng để thực hiện sắp xếp (sorting) và phân trang (pagination) trực tiếp trong các Query Methods hoặc thông qua các tham số `Sort` và `Pageable` .

Java

```
public interface UserRepository extends JpaRepository<User, Long> {

    // Sorting: Sắp xếp kết quả trực tiếp trong tên phương thức.
    List<User> findByStatusOrderByNameAsc(String status); // Tìm User theo
    // trạng thái và sắp xếp theo tên tăng dần.
    List<User> findByStatusOrderByNameAscAgeDesc(String status); // Tìm User
    // theo trạng thái, sắp xếp theo tên tăng dần và tuổi giảm dần.

    // With Sort parameter: Sử dụng đối tượng Sort để sắp xếp.
    List<User> findByStatus(String status, Sort sort); // Tìm User theo trạng
    // thái và áp dụng sắp xếp từ đối tượng Sort.

    // With Pageable: Sử dụng đối tượng Pageable để phân trang và sắp xếp.
    Page<User> findByStatus(String status, Pageable pageable); // Tìm User
    // theo trạng thái và trả về một Page các User.
    Slice<User> findByAgeGreaterThan(Integer age, Pageable pageable); // Tìm
    // User có tuổi lớn hơn và trả về một Slice các User.
}

// Usage examples: Ví dụ về cách sử dụng các phương thức trên trong một
// Service.
@Service
public class UserService {
```

```

@Autowired
private UserRepository userRepository;

public List<User> getActiveUsersSorted() {
    // Tạo đối tượng Sort để sắp xếp theo tên tăng dần, sau đó theo tuổi giảm dần.
    Sort sort = Sort.by(Sort.Direction.ASC, "name")
        .and(Sort.by(Sort.Direction.DESC, "age"));
    return userRepository.findByStatus("ACTIVE", sort); // Gọi phương thức repository với đối tượng Sort.
}

public Page<User> getActiveUsersPage(int page, int size) {
    // Tạo đối tượng Pageable cho trang thứ 'page' với kích thước 'size', sắp xếp theo tên tăng dần.
    Pageable pageable = PageRequest.of(page, size,
        Sort.by("name").ascending());
    return userRepository.findByStatus("ACTIVE", pageable); // Gọi phương thức repository với đối tượng Pageable.
}
}

```

Giải thích:

- **Sắp xếp trong tên phương thức:** Bạn có thể thêm `OrderBy` theo sau là tên thuộc tính và `Asc` (tăng dần) hoặc `Desc` (giảm dần) vào tên phương thức để sắp xếp kết quả. Ví dụ: `OrderByNameAsc`.
- **Sort parameter:** Đối tượng `org.springframework.data.domain.Sort` cho phép bạn định nghĩa các tiêu chí sắp xếp phức tạp hơn, bao gồm sắp xếp theo nhiều thuộc tính và hướng sắp xếp khác nhau. Bạn có thể truyền đối tượng `Sort` này vào phương thức repository.
- **Pageable parameter:** Đối tượng `org.springframework.data.domain.Pageable` là một interface cho phép bạn truyền thông tin về phân trang (số trang, kích thước trang) và sắp xếp vào truy vấn. `PageRequest.of(page, size, sort)` là cách phổ biến để tạo một đối tượng `Pageable`.
- **Page<T> :** Khi sử dụng `Pageable`, phương thức repository sẽ trả về một đối tượng `Page`. `Page` không chỉ chứa danh sách các Entity cho trang hiện tại mà còn cung cấp thông tin về tổng số phần tử, tổng số trang, v.v.

- `Slice<T>` : Tương tự như `Page` , nhưng `Slice` chỉ biết liệu có trang tiếp theo hay không, không biết tổng số trang hoặc tổng số phần tử. Điều này hữu ích cho các trường hợp cần hiệu suất cao hơn khi bạn không cần biết tổng số lượng phần tử.

5.2 @Query Annotation

Sử dụng JPQL hoặc native SQL cho các truy vấn phức tạp.

5.2.1 JPQL Queries

Khi các Query Methods không đủ linh hoạt hoặc quá phức tạp để đặt tên, bạn có thể sử dụng `@Query` annotation để định nghĩa các truy vấn JPQL (Java Persistence Query Language) tùy chỉnh. JPQL là một ngôn ngữ truy vấn hướng đối tượng, tương tự SQL nhưng làm việc với các Entity và thuộc tính của chúng thay vì bảng và cột.

Java

```
public interface UserRepository extends JpaRepository<User, Long> {

    @Query("SELECT u FROM User u WHERE u.email = ?1") // Truy vấn JPQL với
    // tham số vị trí (?1).
    User findByEmailJPQL(String email); // Tìm User theo email sử dụng JPQL.

    @Query("SELECT u FROM User u WHERE u.name = :name AND u.status =
    :status") // Truy vấn JPQL với tham số đặt tên (:name, :status).
    List<User> findByNameAndStatus(@Param("name") String name, // Gán giá trị
    // cho tham số đặt tên "name".
                                   @Param("status") String status); // Gán
    // giá trị cho tham số đặt tên "status".

    @Query("SELECT u FROM User u WHERE u.age BETWEEN :minAge AND :maxAge") //
    // Truy vấn với toán tử BETWEEN và tham số đặt tên.
    List<User> findUsersByAgeRange(@Param("minAge") int minAge, // Tham số
    // cho tuổi tối thiểu.
                                   @Param("maxAge") int maxAge); // Tham số
    // cho tuổi tối đa.

    // Join queries: Truy vấn với các mối quan hệ giữa các Entity.
    @Query("SELECT u FROM User u JOIN u.roles r WHERE r.name = :roleName") //
    // JOIN giữa User và Role.
    List<User> findUsersByRoleName(@Param("roleName") String roleName); //
    // Tìm User theo tên Role.

    @Query("SELECT u FROM User u LEFT JOIN FETCH u.roles WHERE u.id = :id")
```

```
// LEFT JOIN FETCH để tải eagerly các roles cùng với User.
User findUserWithRoles(@Param("id") Long id); // Tìm User và tải luôn các
Role liên quan.

// Aggregate functions: Các hàm tổng hợp như COUNT, AVG, SUM, MIN, MAX.
@Query("SELECT COUNT(u) FROM User u WHERE u.status = :status") // Đếm số
lượng User theo trạng thái.
long countUsersByStatus(@Param("status") String status);

@Query("SELECT AVG(u.age) FROM User u WHERE u.status = 'ACTIVE'") // Tính
tuổi trung bình của User đang hoạt động.
Double getAverageAgeOfActiveUsers();

// Constructor expressions: Trả về một đối tượng DTO (Data Transfer
Object) thay vì Entity.
@Query("SELECT new com.example.dto.UserDTO(u.name, u.email) FROM User u
WHERE u.status = :status") // Tạo một UserDTO mới từ các thuộc tính của User.
List<UserDTO> findUserDTOsByStatus(@Param("status") String status);
}
```

Giải thích:

- **JPQL (Java Persistence Query Language):** Là một ngôn ngữ truy vấn độc lập với cơ sở dữ liệu, được sử dụng để truy vấn các Entity và mối quan hệ của chúng. Nó tương tự SQL nhưng làm việc với các đối tượng Java thay vì bảng và cột.
- **@Query annotation:** Cho phép bạn viết các truy vấn JPQL hoặc Native SQL trực tiếp trong interface repository. Điều này rất hữu ích cho các truy vấn phức tạp mà Query Methods không thể xử lý.
- **Tham số:** Bạn có thể sử dụng tham số vị trí (?1 , ?2 , ...) hoặc tham số đặt tên (:paramName). Sử dụng tham số đặt tên với @Param annotation thường được khuyến nghị vì nó dễ đọc và ít gây lỗi hơn.
- **JOIN FETCH :** Được sử dụng để tải eagerly các mối quan hệ (ví dụ: roles của User) cùng với Entity chính trong một truy vấn duy nhất, giúp tránh vấn đề N+1 select problem.
- **Hàm tổng hợp:** JPQL hỗ trợ các hàm tổng hợp như COUNT , AVG , SUM , MIN , MAX .
- **Constructor expressions:** Cho phép bạn trả về các đối tượng DTO tùy chỉnh thay vì toàn bộ Entity, giúp tối ưu hóa việc truyền dữ liệu và chỉ tải những thông tin cần thiết.

5.2.2 Native SQL Queries

Trong một số trường hợp, bạn có thể cần sử dụng Native SQL (SQL thuần túy) thay vì JPQL, đặc biệt khi bạn muốn tận dụng các tính năng cụ thể của cơ sở dữ liệu hoặc khi truy vấn quá phức tạp để biểu diễn bằng JPQL.

Java

```
public interface UserRepository extends JpaRepository<User, Long> {

    @Query(value = "SELECT * FROM users WHERE email = ?1", nativeQuery =
true) // Truy vấn Native SQL với tham số vị trí.
    User findByEmailNative(String email); // Tìm User theo email sử dụng
Native SQL.

    @Query(value = "SELECT u.* FROM users u WHERE u.created_date > :date",
        nativeQuery = true) // Truy vấn Native SQL với tham số đặt tên.
    List<User> findUsersCreatedAfter(@Param("date") Date date); // Tìm User
được tạo sau một ngày cụ thể.

    // Complex native queries: Ví dụ về một truy vấn Native SQL phức tạp.
    @Query(value = ""
        SELECT u.*, COUNT(o.id) as order_count
        FROM users u
        LEFT JOIN orders o ON u.id = o.user_id
        WHERE u.status = :status
        GROUP BY u.id
        HAVING COUNT(o.id) > :minOrders
        "", nativeQuery = true) // Sử dụng block text "" để viết truy vấn
SQL dài và dễ đọc.
    List<Object[]> findActiveUsersWithMinOrders(@Param("status") String
status, // Tham số cho trạng thái User.
                                                @Param("minOrders") int
minOrders); // Tham số cho số lượng đơn hàng tối thiểu.
}
```

Giải thích:

- **nativeQuery = true** : Thuộc tính này trong `@Query` annotation báo cho Spring Data JPA biết rằng đây là một truy vấn Native SQL, không phải JPQL.
- **Ưu điểm**: Cho phép bạn sử dụng toàn bộ sức mạnh của SQL và các tính năng cụ thể của cơ sở dữ liệu (ví dụ: các hàm riêng, cú pháp đặc biệt).

- **Nhược điểm:** Truy vấn Native SQL không độc lập với cơ sở dữ liệu. Nếu bạn thay đổi loại cơ sở dữ liệu, bạn có thể phải viết lại các truy vấn này.
- **Trả về `Object[]`** : Khi truy vấn Native SQL trả về các cột không ánh xạ trực tiếp tới một Entity, kết quả thường được trả về dưới dạng `List<Object[]>` hoặc `List<Map<String, Object>>` . Bạn sẽ cần tự xử lý các kết quả này.

5.2.3 @Modifying Queries

Khi bạn muốn thực hiện các thao tác thay đổi dữ liệu (INSERT, UPDATE, DELETE) bằng JPQL hoặc Native SQL thông qua `@Query` annotation, bạn cần đánh dấu phương thức đó bằng `@Modifying` . Điều này báo cho Spring Data JPA biết rằng phương thức này sẽ thay đổi trạng thái của cơ sở dữ liệu.

Java

```
public interface UserRepository extends JpaRepository<User, Long> {

    @Modifying // Báo hiệu đây là một truy vấn thay đổi dữ liệu.
    @Query("UPDATE User u SET u.status = :status WHERE u.id = :id") // Truy
    vấn JPQL để cập nhật trạng thái User.
    int updateUserStatus(@Param("id") Long id, @Param("status") String
    status); // Trả về số lượng bản ghi đã được cập nhật.

    @Modifying
    @Query("UPDATE User u SET u.lastLoginDate = :date WHERE u.email =
    :email") // Truy vấn JPQL để cập nhật ngày đăng nhập cuối cùng.
    void updateLastLoginDate(@Param("email") String email, @Param("date")
    Date date); // Không trả về giá trị.

    @Modifying
    @Query("DELETE FROM User u WHERE u.status = :status") // Truy vấn JPQL để
    xóa User theo trạng thái.
    void deleteUsersByStatus(@Param("status") String status); // Không trả về
    giá trị.

    // Native modifying queries: Tương tự với Native SQL.
    @Modifying
    @Query(value = "UPDATE users SET status = 'INACTIVE' WHERE
    last_login_date < :date",
        nativeQuery = true) // Truy vấn Native SQL để cập nhật trạng thái.
    int deactivateInactiveUsers(@Param("date") Date date); // Trả về số lượng
    bản ghi đã được cập nhật.
}
```

```
// Usage in Service: Cách sử dụng trong lớp Service.
@Service
@Transactional // Đảm bảo phương thức này chạy trong một transaction.
public class UserService {

    @Autowired
    private UserRepository userRepository;

    public void updateUserStatus(Long userId, String status) {
        int updatedRows = userRepository.updateUserStatus(userId, status); //
        Gọi phương thức cập nhật.
        if (updatedRows == 0) {
            throw new EntityNotFoundException("User not found"); // Xử lý
            trường hợp không tìm thấy User.
        }
    }
}
```

Giải thích:

- **@Modifying** : Bắt buộc phải có khi bạn sử dụng **@Query** để thực hiện các thao tác **INSERT** , **UPDATE** , **DELETE** . Nếu không có annotation này, Spring Data JPA sẽ coi đây là một truy vấn **SELECT** và sẽ ném ra lỗi.
- **Trả về giá trị**: Các phương thức được đánh dấu **@Modifying** có thể trả về **void** hoặc **int** . Nếu trả về **int** , nó sẽ là số lượng bản ghi đã bị ảnh hưởng bởi thao tác (số lượng hàng được cập nhật hoặc xóa).
- **Transaction**: Các thao tác thay đổi dữ liệu phải luôn được thực hiện trong một transaction. Đảm bảo rằng phương thức gọi repository (thường là trong lớp Service) được đánh dấu bằng **@Transactional** .

5.3 Named Queries

Named Queries là các truy vấn JPQL hoặc Native SQL được định nghĩa trước và đặt tên. Chúng có thể được định nghĩa trong Entity bằng các annotation hoặc trong file XML. Việc sử dụng Named Queries giúp tập trung các truy vấn, cải thiện khả năng đọc và tái sử dụng code.

5.3.1 Named Queries trong Entity

Bạn có thể định nghĩa Named Queries trực tiếp trong lớp Entity bằng cách sử dụng `@NamedQuery` hoặc `@NamedQueries` (nếu có nhiều truy vấn).

Java

```
@Entity
@NamedQueries({ // Sử dụng @NamedQueries để định nghĩa nhiều Named Query.
    @NamedQuery( // Định nghĩa một Named Query.
        name = "User.findByStatus", // Tên của Named Query. Quy ước thường là
        EntityName.methodName.
        query = "SELECT u FROM User u WHERE u.status = :status" // Truy vấn
        JPQL.
    ),
    @NamedQuery(
        name = "User.findActiveUsersWithRoles",
        query = "SELECT u FROM User u JOIN FETCH u.roles WHERE u.status =
        'ACTIVE'"
    )
})
public class User {
    // Entity fields and methods
}

// Repository usage: Cách sử dụng Named Queries trong Repository.
public interface UserRepository extends JpaRepository<User, Long> {

    // Spring Data JPA tự động tìm named query có tên khớp với phương thức.
    // Tên phương thức phải khớp với tên của Named Query (ví dụ:
    findByStatus).
    List<User> findByStatus(@Param("status") String status);

    List<User> findActiveUsersWithRoles();
}
```

Giải thích:

- `@NamedQuery` : Annotation này được đặt trên lớp Entity để định nghĩa một truy vấn JPQL được đặt tên.
- `name` : Tên duy nhất của truy vấn. Spring Data JPA có thể tự động liên kết các phương thức repository với Named Queries nếu tên phương thức khớp với tên của Named Query (ví dụ: `findByStatus` trong repository sẽ tìm Named Query có tên `User.findByStatus`).
- `query` : Chuỗi JPQL của truy vấn.

- **Lợi ích:** Tách biệt truy vấn khỏi code Java, giúp dễ dàng quản lý và thay đổi truy vấn mà không cần biên dịch lại code. Cải thiện khả năng đọc và tái sử dụng.

5.3.2 Named Native Queries

Tương tự như Named Queries cho JPQL, bạn cũng có thể định nghĩa Named Native Queries (truy vấn SQL thuần túy được đặt tên) trong Entity bằng cách sử dụng `@NamedNativeQuery` hoặc `@NamedNativeQueries` .

Java

```
@Entity
@NamedNativeQueries({ // Sử dụng @NamedNativeQueries để định nghĩa nhiều
    Named Native Query.
        @NamedNativeQuery( // Định nghĩa một Named Native Query.
            name = "User.findByEmailNative", // Tên của Named Native Query.
            query = "SELECT * FROM users WHERE email = :email", // Truy vấn
Native SQL.
            resultClass = User.class // Chỉ định lớp Entity mà kết quả truy vấn
sẽ được ánh xạ tới.
        )
    })
public class User {
    // Entity definition
}
```

Giải thích:

- `@NamedNativeQuery` : Annotation này được đặt trên lớp Entity để định nghĩa một truy vấn Native SQL được đặt tên.
- `name` : Tên duy nhất của truy vấn Native SQL.
- `query` : Chuỗi Native SQL của truy vấn.
- `resultClass` : Thuộc tính này rất quan trọng. Nó chỉ định lớp Entity mà kết quả của truy vấn Native SQL sẽ được ánh xạ tới. Nếu bạn không chỉ định `resultClass` , kết quả sẽ được trả về dưới dạng `Object[]` hoặc `Map<String, Object>` .
- **Lợi ích:** Giống như Named Queries cho JPQL, Named Native Queries giúp tập trung các truy vấn SQL, cải thiện khả năng đọc và tái sử dụng. Chúng cũng hữu ích khi bạn cần sử dụng

dụng các tính năng SQL cụ thể của cơ sở dữ liệu mà JPQL không hỗ trợ.

5.4 Specifications (Criteria API)

Specifications là một cách mạnh mẽ để xây dựng các truy vấn động (dynamic queries) và có thể tái sử dụng trong Spring Data JPA. Chúng dựa trên JPA Criteria API, cho phép bạn xây dựng các truy vấn một cách lập trình (programmatically) thay vì viết chuỗi truy vấn tĩnh. Điều này đặc biệt hữu ích khi bạn có nhiều điều kiện tìm kiếm tùy chọn.

Java

```
public class UserSpecifications {

    // Specification để tìm User theo tên. Nếu tên là null, trả về null
    // (không áp dụng điều kiện).
    public static Specification<User> hasName(String name) {
        return (root, query, criteriaBuilder) ->
            name == null ? null : criteriaBuilder.equal(root.get("name"),
name);
    }

    // Specification để tìm User theo trạng thái.
    public static Specification<User> hasStatus(String status) {
        return (root, query, criteriaBuilder) ->
            status == null ? null : criteriaBuilder.equal(root.get("status"),
status);
    }

    // Specification để tìm User có tuổi lớn hơn một giá trị nhất định.
    public static Specification<User> ageGreaterThan(Integer age) {
        return (root, query, criteriaBuilder) ->
            age == null ? null : criteriaBuilder.greaterThan(root.get("age"),
age);
    }

    // Specification để tìm User có một Role cụ thể.
    public static Specification<User> hasRole(String roleName) {
        return (root, query, criteriaBuilder) -> {
            if (roleName == null) return null;

            Join<User, Role> roleJoin = root.join("roles"); // Thực hiện JOIN
với bảng roles.
            return criteriaBuilder.equal(roleJoin.get("name"), roleName); //
Điều kiện so sánh tên role.
        };
    }
}
```

```

    }
}

// Repository: Interface repository cần kế thừa JpaSpecificationExecutor để
// sử dụng Specifications.
public interface UserRepository extends JpaRepository<User, Long>,
JpaSpecificationExecutor<User> {
    // JpaSpecificationExecutor cung cấp các phương thức như:
    // Optional<T> findOne(Specification<T> spec);
    // List<T> findAll(Specification<T> spec);
    // Page<T> findAll(Specification<T> spec, Pageable pageable);
    // List<T> findAll(Specification<T> spec, Sort sort);
    // long count(Specification<T> spec);
}

// Service usage: Cách sử dụng Specifications trong lớp Service.
@Service
public class UserService {

    @Autowired
    private UserRepository userRepository;

    public List<User> findUsers(String name, String status, Integer minAge,
String roleName) {
        // Khởi tạo một Specification rỗng hoặc với điều kiện mặc định.
        Specification<User> spec = Specification.where(null);

        // Thêm các điều kiện vào Specification nếu chúng không null.
        if (name != null) {
            spec = spec.and(UserSpecifications.hasName(name));
        }
        if (status != null) {
            spec = spec.and(UserSpecifications.hasStatus(status));
        }
        if (minAge != null) {
            spec = spec.and(UserSpecifications.ageGreaterThan(minAge));
        }
        if (roleName != null) {
            spec = spec.and(UserSpecifications.hasRole(roleName));
        }

        return userRepository.findAll(spec); // Thực hiện truy vấn với
        Specification đã xây dựng.
    }
}

```

Giải thích:

- **Specification<T>** : Là một interface trong Spring Data JPA cho phép bạn định nghĩa các tiêu chí truy vấn. Mỗi **Specification** đại diện cho một điều kiện **WHERE** .
- **JpaSpecificationExecutor<T>** : Interface này cung cấp các phương thức để thực thi các truy vấn dựa trên **Specification** . Repository của bạn cần kế thừa interface này để sử dụng Specifications.
- **toPredicate(Root<T> root, CriteriaQuery<?> query, CriteriaBuilder criteriaBuilder)** : Đây là phương thức chính của interface **Specification** . Nó nhận vào **Root** (đại diện cho Entity gốc), **CriteriaQuery** (để xây dựng truy vấn) và **CriteriaBuilder** (để xây dựng các biểu thức điều kiện).
- **Specification.where(null)** : Khởi tạo một **Specification** rỗng. Bạn có thể bắt đầu với **null** và sau đó thêm các điều kiện bằng cách sử dụng **and()** hoặc **or()** .
- **spec.and(...)** / **spec.or(...)** : Cho phép bạn kết hợp nhiều **Specification** với nhau bằng toán tử **AND** hoặc **OR** .
- **Lợi ích:** Specifications rất hữu ích cho việc xây dựng các truy vấn động, phức tạp, và có thể tái sử dụng. Chúng giúp tách biệt logic xây dựng truy vấn khỏi logic nghiệp vụ, làm cho code sạch hơn và dễ bảo trì hơn.

5.5 Projections

Projections trong Spring Data JPA cho phép bạn chỉ chọn một tập hợp con các thuộc tính của một Entity thay vì tải toàn bộ Entity. Điều này rất hữu ích để tối ưu hóa hiệu suất bằng cách giảm lượng dữ liệu được truy xuất từ cơ sở dữ liệu và truyền qua mạng.

5.5.1 Interface-based Projections

Đây là cách đơn giản nhất để tạo Projections. Bạn định nghĩa một interface với các getter cho các thuộc tính mà bạn muốn chọn. Spring Data JPA sẽ tự động tạo ra một implementation của interface này.

Java

```
// Projection interface: Định nghĩa các getter cho các thuộc tính muốn lấy.
public interface UserSummary {
    String getName(); // Lấy thuộc tính name của User.
```



```

String getEmail(); // Lấy thuộc tính email của User.

// Nested projection: Bạn có thể lồng các Projection interface vào nhau.
AddressSummary getAddress(); // Lấy thông tin địa chỉ dưới dạng
AddressSummary.

interface AddressSummary {
    String getCity(); // Lấy thuộc tính city của Address.
    String getCountry(); // Lấy thuộc tính country của Address.
}

// Repository: Cách sử dụng Interface-based Projections trong Repository.
public interface UserRepository extends JpaRepository<User, Long> {

    // Spring Data JPA tự động tạo truy vấn để chỉ chọn các trường trong
    UserSummary.
    List<UserSummary> findByStatus(String status);

    // Bạn cũng có thể sử dụng @Query để định nghĩa rõ ràng các trường cần
    chọn.
    @Query("SELECT u.name as name, u.email as email FROM User u WHERE
    u.status = :status")
    List<UserSummary> findUserSummariesByStatus(@Param("status") String
    status);
}

```

Giải thích:

- **Interface-based:** Bạn chỉ cần định nghĩa một interface với các phương thức getter tương ứng với các thuộc tính của Entity mà bạn muốn truy xuất. Tên phương thức getter phải tuân theo quy ước JavaBean (ví dụ: `getName()` cho thuộc tính `name`).
- **Nested Projections:** Bạn có thể lồng các Projection interface vào nhau để truy xuất các thuộc tính từ các Entity liên quan.
- **Sử dụng trong Repository:** Spring Data JPA sẽ tự động nhận diện các phương thức trả về Projection interface và tạo truy vấn phù hợp để chỉ chọn các cột cần thiết.
- **Với `@Query` :** Bạn có thể sử dụng `@Query` để định nghĩa rõ ràng các cột cần chọn và ánh xạ chúng tới các phương thức getter trong Projection interface. Điều quan trọng là alias của cột trong truy vấn phải khớp với tên thuộc tính trong interface (ví dụ: `u.name as name`).

5.5.2 Class-based Projections (DTOs)

Class-based Projections cho phép bạn ánh xạ kết quả truy vấn trực tiếp vào các đối tượng DTO (Data Transfer Object) tùy chỉnh. Điều này hữu ích khi bạn cần thêm logic hoặc các trường tính toán vào đối tượng kết quả.

Java

```
// DTO class: Lớp DTO cần có constructor phù hợp với các trường được chọn
// trong truy vấn.
public class UserDTO {
    private String name;
    private String email;

    // Constructor phải khớp với thứ tự và kiểu dữ liệu của các trường trong
    // truy vấn JPQL.
    public UserDTO(String name, String email) {
        this.name = name;
        this.email = email;
    }

    // getters and setters (cần thiết để truy cập dữ liệu từ DTO)
    public String getName() {
        return name;
    }

    public void setName(String name) {
        this.name = name;
    }

    public String getEmail() {
        return email;
    }

    public void setEmail(String email) {
        this.email = email;
    }
}

// Repository: Cách sử dụng Class-based Projections trong Repository.
public interface UserRepository extends JpaRepository<User, Long> {

    // Sử dụng từ khóa NEW trong JPQL để gọi constructor của DTO.
    @Query("SELECT new com.example.dto.UserDTO(u.name, u.email) FROM User u
    WHERE u.status = :status")
}
```

```
List<UserDTO> findUserDTOsByStatus(@Param("status") String status);  
}
```

Giải thích:

- **DTO Class:** Lớp DTO cần có một constructor với các tham số khớp với thứ tự và kiểu dữ liệu của các trường được chọn trong truy vấn JPQL. Các getter và setter cũng cần được cung cấp để truy cập dữ liệu.
- **NEW keyword trong JPQL:** Để sử dụng Class-based Projections, bạn cần sử dụng từ khóa **NEW** trong truy vấn JPQL, theo sau là tên đầy đủ của lớp DTO và các tham số constructor (các trường bạn muốn chọn từ Entity).
- **Lợi ích:** Cung cấp sự linh hoạt hơn so với Interface-based Projections vì bạn có thể thêm logic nghiệp vụ, các trường tính toán, hoặc các phương thức tiện ích vào lớp DTO. Điều này giúp tạo ra các đối tượng kết quả giàu tính năng hơn.

5.5.3 Dynamic Projections

Dynamic Projections cho phép bạn quyết định loại Projection (interface-based hoặc class-based) tại thời điểm chạy (runtime). Điều này rất linh hoạt khi bạn muốn sử dụng cùng một phương thức repository để trả về các loại Projection khác nhau tùy thuộc vào ngữ cảnh.

Java

```
public interface UserRepository extends JpaRepository<User, Long> {  
  
    // Phương thức này có thể trả về bất kỳ loại Projection nào (T) được chỉ định.  
    <T> List<T> findByStatus(String status, Class<T> type);  
}  
  
// Usage: Cách sử dụng Dynamic Projections.  
@Service  
public class UserService {  
  
    @Autowired  
    private UserRepository userRepository;  
  
    public List<UserSummary> getActiveUserSummaries() {  
        // Trả về danh sách UserSummary (Interface-based Projection).  
        return userRepository.findByStatus("ACTIVE", UserSummary.class);  
    }  
}
```

```

    }

    public List<UserDTO> getActiveUserDTOs() {
        // Trả về danh sách UserDTO (Class-based Projection).
        return userRepository.findByStatus("ACTIVE", UserDTO.class);
    }

    public List<User> getActiveUsers() {
        // Trả về danh sách Entity đầy đủ nếu bạn muốn.
        return userRepository.findByStatus("ACTIVE", User.class);
    }
}

```

Giải thích:

- **Tham số `Class<T> type`** : Phương thức repository nhận một tham số `Class<T> type` , cho phép bạn truyền vào lớp hoặc interface của Projection mà bạn muốn nhận về.
- **Tính linh hoạt:** Cùng một phương thức `findByStatus` có thể được sử dụng để lấy về `UserSummary` , `UserDTO` , hoặc thậm chí là Entity `User` đầy đủ, tùy thuộc vào tham số `type` được truyền vào.
- **Hạn chế:** Dynamic Projections chỉ hoạt động với các phương thức Query Methods (Derived Queries) hoặc các truy vấn `@Query` đơn giản. Đối với các truy vấn phức tạp hơn hoặc Native SQL, bạn có thể cần phải tự xử lý ánh xạ kết quả.

5.6 Query by Example

Query by Example (QBE) là một tính năng của Spring Data JPA cho phép bạn xây dựng các truy vấn động dựa trên một đối tượng "probe" (ví dụ). Nó rất hữu ích cho các trường hợp tìm kiếm đơn giản, không yêu cầu các điều kiện phức tạp như JOIN hoặc GROUP BY.

Java

```

@Service
public class UserService {

    @Autowired
    private UserRepository userRepository;

    public List<User> findUsersByExample(String name, String status) {
        // 1. Tạo đối tượng Probe (Example Entity): Một instance của Entity
    }
}

```

chứa các giá trị mà bạn muốn tìm kiếm.

```
User probe = new User();
probe.setName(name); // Đặt giá trị cho các thuộc tính bạn muốn dùng
                        // làm điều kiện tìm kiếm.
probe.setStatus(status);

// 2. Tạo ExampleMatcher: Định nghĩa cách các thuộc tính sẽ được so
sánh.
ExampleMatcher matcher = ExampleMatcher.matching() // Bắt đầu với một
matcher mặc định.
    .withIgnoreCase() // Bỏ qua chữ hoa/thường cho tất cả các thuộc
tính kiểu String.
    .withStringMatcher(ExampleMatcher.StringMatcher.CONTAINING); //
Sử dụng toán tử LIKE %value% cho tất cả các thuộc tính kiểu String.

// Bạn cũng có thể tùy chỉnh cho từng thuộc tính:
// ExampleMatcher matcher = ExampleMatcher.matching()
//     .withMatcher("name",
ExampleMatcher.GenericPropertyMatchers.startsWith().ignoreCase())
//     .withMatcher("email",
ExampleMatcher.GenericPropertyMatchers.endsWith());

// 3. Tạo Example: Kết hợp probe và matcher.
Example<User> example = Example.of(probe, matcher);

// 4. Thực hiện truy vấn: Sử dụng phương thức findAll(Example) của
JpaRepository.
return userRepository.findAll(example);
}
}
```

Giải thích:

- **Example<T>** : Đối tượng chính trong Query by Example. Nó bao gồm hai phần:
 - **Probe**: Một instance của Entity (ví dụ: `User`) với các thuộc tính được đặt giá trị mà bạn muốn tìm kiếm. Các thuộc tính có giá trị `null` sẽ bị bỏ qua trong truy vấn.
 - **ExampleMatcher** : Định nghĩa cách các thuộc tính trong probe sẽ được so sánh. Bạn có thể cấu hình các tùy chọn như bỏ qua chữ hoa/thường (`withIgnoreCase()`), cách so sánh chuỗi (`withStringMatcher()`), hoặc bỏ qua các thuộc tính cụ thể (`ignorePaths()`).
- **ExampleMatcher.matching()** : Tạo một `ExampleMatcher` mặc định, trong đó tất cả các thuộc tính kiểu chuỗi sẽ được so sánh bằng cách khớp chính xác (`StringMatcher.DEFAULT`)

và phân biệt chữ hoa/thường.

- **withIgnoreCase()** : Áp dụng việc bỏ qua chữ hoa/thường cho tất cả các thuộc tính kiểu chuỗi.
- **withStringMatcher()** : Định nghĩa cách các thuộc tính kiểu chuỗi được so sánh. Các tùy chọn phổ biến:
 - **DEFAULT** : Khớp chính xác.
 - **CONTAINING** : Tương đương **LIKE %value%** .
 - **STARTING** : Tương đương **LIKE value%** .
 - **ENDING** : Tương đương **LIKE %value** .
- **findAll(Example<T> example)** : Phương thức này được cung cấp bởi **QueryByExampleExecutor** (mà **JpaRepository** kế thừa) để thực thi truy vấn dựa trên **Example** .
- **Lợi ích**: QBE đơn giản hóa việc xây dựng các truy vấn động cho các trường hợp tìm kiếm cơ bản, giảm thiểu boilerplate code. Nó đặc biệt hữu ích cho các form tìm kiếm đơn giản trên giao diện người dùng.